

Diário de Pesquisa: Projeto FedCycle

Autor: Emmanoel Barbosa

Orientador: Prof. Bruno José Torres Fernandes (Universidade de Pernambuco)

Tema: Um Ciclo de Aprendizado por Destilação para Coevolução de Modelos em Redes Federadas Heterogêneas.

Data da última atualização: Março de 2026

Visão Geral do Projeto (O "Modelo Paulo Freire" de I.A.)

O **FedCycle** é uma arquitetura cíclica de *Federated Learning* (FL) baseada em destilação de conhecimento, inspirada no algoritmo FedBKD. O projeto visa resolver a queda de desempenho em cenários com restrição de hardware (TinyML/ESP32) e dados Não-IID.

Utilizamos a analogia da pedagogia freireana:

1. **O Servidor (Professor):** Possui alta capacidade computacional e extrai características gerais (Camada de Representação). Ele ensina essa base para o cliente.
 2. **O Cliente ESP32 (Aluno):** Possui severas restrições de memória, mas detém a vivência do mundo real (Dados Locais Não-IID). Ele atualiza sua Camada de Classificação e "ensina" de volta o servidor, melhorando a generalização global sem expor dados brutos.
-

✓ Marcos Alcançados (Progresso Técnico)

1. Preparação de Dados Não-IID

- **Script:** `datasetConf.py`
- **Status:** Concluído ✓
- **Descrição:** * Download automático do dataset **CIFAR-10** (usando `torchvision`).
 - Implementação da técnica de "Shards" para dividir os dados de forma estritamente Não-IID (cada cliente simulado recebe apenas 2 classes de imagens).
 - Criação da função de exportação de dados para o simulador **Wokwi**: conversão de tensores de imagem (32x32 RGB) para matrizes unidimensionais nativas de C++, gerando o arquivo `dados_cliente0.h` (respeitando os limites da memória SRAM do ESP32).

2. Separação da Arquitetura de Camadas

- **Script:** `fedcycle_model.py` (ou `modelo_fedcycle.py`)

- **Status:** Concluído ✓
- **Descrição:** * Construção da rede neural em **PyTorch**.
 - Divisão lógica inspirada no FedRep/FedBKD:
 - **Representação (Theta R)** : Camadas convolucionais encarregadas de extrair *feature maps* (tamanho 2048). Será mantida congelada no cliente.
 - **Classificação (Theta C)** : Camadas densas lineares. É a única parte que será treinada via Gradiente Descendente (*On-Device Training*) no microcontrolador.

3. Gerador Data-Free (A GAN de Privacidade)

- **Script:** `gan_generator.py`
- **Status:** Concluído ✓
- **Descrição:** * Implementação da Rede Generativa Adversarial que opera no servidor central.
 - O Gerador recebe ruído aleatório e cria *Feature Maps* sintéticos (vetores de 2048 atributos) para evitar o tráfego de imagens reais, preservando a privacidade.
 - **Funções de Perda Implementadas:**
 1. **Confiança (L_{oh} - Cross-Entropy):** Força o gerador a criar dados que o modelo do cliente reconheça com convicção.
 2. **Diversidade (L_{ms}):** Pune matematicamente o gerador se ele produzir mapas de características muito semelhantes entre si.

4. O Ciclo de Destilação Bidirecional

- **Script:** `bidirectional_distillation.py`
- **Status:** Concluído ✓
- **Descrição:** * Implementação da **Divergência Kullback-Leibler (KL)** para alinhar as distribuições matemáticas entre os modelos.
 - **Global $\rightarrow$ Local:** O Servidor transfere seu conhecimento de representação para o Cliente usando os dados sintéticos da GAN.
 - **Local $\rightarrow$ Global:** O Cliente usa os mesmos dados sintéticos para transferir seu aprendizado empírico de volta ao Servidor, mitigando o esquecimento catastrófico do modelo global.

5. Exportação para TinyML (Ponte para o Wokwi)

- **Script:** `export_tinyML.py`
- **Status:** Concluído (Fase ONNX) ✓
- **Descrição:** * Instalação das bibliotecas `onnx` e `onnxscript`.
 - Extração e desmembramento do modelo treinado em Python para o formato universal ONNX.
 - Geração dos arquivos:
 - `representacao_congelada.onnx` (Pronto para virar `.tflite`).
 - `classificacao_treinavel.onnx` (Pronto para extração de pesos).



Próximos Passos (To-Do)

1. **Missão A (Inferência no Wokwi):** Converter o arquivo `representacao_congelada.onnx` para `.tflite` com quantização INT8, e transformá-lo em um array C++ (`modelo_representacao.h`).
 2. **Missão B (Treinamento On-Device):** Escrever um script Python que extrai as matrizes de Pesos (*Weights*) e Viéses (*Biases*) de `classificacao_treinavel.onnx` e exporta para C++.
 3. **Missão C (C++ no ESP32):** Implementar a matemática de Inferência (TensorFlow Lite Micro) integrada com a matemática de Treinamento (Retropropagação manual da Camada de Classificação) no ambiente Wokwi.
 4. **Missão D (Integração de Redes):** Fechar o ciclo enviando os novos pesos do Wokwi de volta ao Servidor Python usando MQTT ou HTTP via WiFi.
-

Fim do log. Projeto progredindo em ritmo excelente.